

BANKON ENS Subname Registrar — Production-Grade Technical Architecture for `bankon.eth`

TL;DR

The BANKON ENS subname registrar should be built as an **L1-native NameWrapper-anchored system** centered on `bankon.eth` registered through the current `ETHRegistrarController` at `0xfb3cE5D01e0f33f41DbB39035dB9745962F1f968`, wrapped into the `NameWrapper` at `0x0635513f179D50A207757E05759CbD106d7dFcE8`, and exposed through a custom `BankonSubnameRegistrar.sol` that mints `alice.bankon.eth` subnames as ERC-1155 wrapped fuses-locked tokens — accepting tri-rail settlement (native ETH on L1, USDC via permit, and Algorand USDC-via-x402 cross-chain receipts using the GoPlausible `x402-avm` mechanism at ASA ID 31566704). ENS Labs publicly cancelled its Namechain L2 in February 2026 and is consolidating ENSv2 on L1, so the optimal architecture for May 2026 is L1-primary with optional Durin-style CCIP-Read L2 mirroring (registry-factory `0xDdddDdDDD8Aa1f237b4fa0669cb46892346d22d`) for high-volume issuance. [The Block](#) [GitHub](#)

- **`bankon.eth` is a 6-character name priced at \$5/year via the ENS StablePriceOracle** (paid in ETH at the Chainlink ETH/USD rate); registration uses the commit→reveal flow with `MIN_REGISTRATION_DURATION = 28 days`, a 60-second minimum commitment age, and a 24-hour maximum. [Etherscan + 2](#)
- **Subname economics:** BankonSubnameRegistrar mints wrapped ERC-1155 subnames with `PARENT_CANNOT_CONTROL | CANNOT_UNWRAP` burned for "unruggable" guarantees; fee splits route to a BANKON treasury Safe, an optional AgenticPlace listing pool, and (for the BankonDomainHosting meta-contract) the external parent-domain owner.
- **iNFT / agent identity:** each subname is optionally wrapped under ERC-7857 (OG Labs Intelligent NFT standard) with an ERC-6551 token-bound account derived through the canonical singleton registry at `0x0000000006551c19487814612e58FE06813775758`, giving every `agent.bankon.eth` an autonomous wallet, encrypted-metadata pointer, and `mindX` API endpoint addressable through a TEXT record on the BankonSubnameResolver. [0g AI](#) [Tokenbound](#)

Key Findings

Architectural verdict: L1-primary, with optional L2 mirror

The single most important contextual fact is that **ENS Labs announced on February 6, 2026 that Namechain (its planned ENS-specific L2) is cancelled** and ENSv2 will deploy exclusively on Ethereum mainnet. Nick Johnson's blog post ([ens.domains/blog/post/ens-staying-on-](#)

ethereum, February 6, 2026) explained the reasoning bluntly: "Ethereum is scaling faster than almost anyone predicted two years ago; we've seen a 99% reduction in ENS registration gas costs over the past year, coinciding with Ethereum's gas limit increases from 30M to 60M in 2025." The same post quantifies the change exactly: "ENS name registration now costs less than 5 cents on average in gas, down from close to \$5 in gas a year ago." This collapses the rationale for building a custom-L2 subname registrar. The right BANKON design is therefore L1-primary, with a clearly documented Durin-style L2 fallback path that can be flipped on if BANKON volume exceeds the threshold where per-mint L1 gas exceeds the per-mint BANKON fee. (BitcoinEthereumNews.com...)

The complete L1 contract stack relevant to `bankon.eth`

The canonical ENS contract deployments are tracked in the [ensdomains/ens-contracts wiki](#), most recently edited March 4, 2026. The contracts BANKON's registrar must integrate with are:

- **ENS Registry** at `0x00000000000000C2E074eC69A0dFb2997BA6C7d2e1e` — source `contracts/registry/ENSRegistry.sol`. Exposes `owner(bytes32 node)`, `setSubnodeOwner(bytes32 node, bytes32 label, address owner)`, `setResolver(bytes32 node, address resolver)`, and `setRecord(bytes32 node, address owner, address resolver, uint64 ttl)`. All subname creation that bypasses the NameWrapper would happen here, but BANKON should not bypass the wrapper. (GitHub) (Etherscan)
- **BaseRegistrarImplementation** (the .eth ERC-721) at `0x57f1887a8BF19b14fC0dF6Fd9B2acc9Af147eA85` — source `contracts/ethregistrar/BaseRegistrarImplementation.sol`. Holds `expiries[id]`, exposes `nameExpires(uint256 id)`, `register(uint256 id, address owner, uint duration)`, `renew(uint256 id, uint duration)`, `reclaim(uint256 id, address owner)`, and the hard-coded `GRACE_PERIOD = 90 days`. The base node hash for `.eth` is `0x93cdeb708b7545dc668eb9280176169d1c33cfd8ed6f04690a0bcc88a93fc4ae`. (GitHub + 5)
- **ETHRegistrarController (current V3)** at `0xfb3cE5D01e0f33f41DbB39035dB9745962F1f968` — source `contracts/ethregistrar/ETHRegistrarController.sol` on the `staging` branch. This is the controller BANKON will use to actually register `bankon.eth`. Its constructor now takes both `IReverseRegistrar` and `IDefaultReverseRegistrar`, and the emitted `NameRegistered(string label, bytes32 indexed labelhash, address indexed owner, uint256 baseCost, uint256 premium, uint256 expires, bytes32 referrer)` event includes a `referrer` field. The legacy controllers `0x253553366Da8546fC250F225fe3d25d0C782303b`, `0x7e02892cfc2Bfd53a75275451d73cF620e793fc0`, `0xf0ad5cad05e10572efceb849f6ff0c68f9700455`, and `0x283af0b28c62c092c9727f1ee09c02ca627eb7f5` still exist but should be ignored. The `MIN_REGISTRATION_DURATION` constant is `28 days`. The hard-coded `ETH_NODE` is `0x93cdeb708b7545dc668eb9280176169d1c33cfd8ed6f04690a0bcc88a93fc4ae`. (GitHub + 5)

- **NameWrapper** at [0x0635513f179D50A207757E05759CbD106d7dFcE8](#) — source [contracts/wrapper/NameWrapper.sol](#). This is the address from the official wiki published March 2026; an older [0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401](#) appeared in the EP3.5 governance proposal but the wiki now shows the redeployed instance. NameWrapper exposes the entire fuses surface area for BANKON, including [wrapETH2LD\(string label, address wrappedOwner, uint16 ownerControlledFuses, address resolver\)](#), [wrap\(bytes name, address wrappedOwner, address resolver\)](#), [setSubnodeOwner\(bytes32 parentNode, string label, address owner, uint32 fuses, uint64 expiry\)](#), [setSubnodeRecord\(bytes32 parentNode, string label, address owner, address resolver, uint64 ttl, uint32 fuses, uint64 expiry\)](#), [setChildFuses\(bytes32 parentNode, bytes32 labelhash, uint32 fuses, uint64 expiry\)](#), and [setFuses\(bytes32 node, uint16 ownerControlledFuses\)](#). [ENS Documentation](#)
- **PublicResolver** at [0x231b0Ee14048e9dCcD1d247744d114a4EB5E8E63](#) — source [contracts/resolvers/PublicResolver.sol](#). Provides [setAddr\(bytes32 node, address\)](#), [setAddr\(bytes32 node, uint256 coinType, bytes\)](#) for ENSIP-9/11 multichain addresses, [setText\(bytes32 node, string key, string value\)](#), [setContenthash\(bytes32 node, bytes hash\)](#), [setName](#), [setABI](#), and the [multicallWithNodeCheck](#) used by the controller to atomically pre-populate records during registration. [Etherscan](#) [LinkedIn](#)
- **UniversalResolver (ENSIP-23)** at [0xEeEeEeE14D718C2B47D9923Deab1335E144EeEe](#) — a proxy owned by the ENS DAO. Its [resolve\(bytes name, bytes data\)](#) returns [\(bytes result, address resolver\)](#) and [reverse\(bytes lookupAddress, uint256 coinType\)](#) are how all modern clients (viem, wagmi, ethers v6) discover BANKON subnames including ones served via CCIP-Read. [Ens](#) [Ens](#)
- **ReverseRegistrar** at [0xa58E81fe9b61B5c3fE2AFD33CF304c454AbFc7Cb](#) and **DefaultReverseRegistrar** at [0x283F227c4Bd38ecE252C4Ae7ECE650B0e913f1f9](#) — for setting [addr.reverse](#) and the new ENSIP-19 default reverse. [GitHub](#)
- **ExponentialPremiumPriceOracle** at [0x6810dbce73c67506f785a225f818b30d8f209aab](#) — source [contracts/ethregistrar/ExponentialPremiumPriceOracle.sol](#). The starting premium is \$100,000,000 and decays exponentially (halving daily) to ~\$0 over 21 days; the floor after day 21 is roughly \$0.048. [GitHub](#) [ENS](#)
- **DNSRegistrar** at [0xB32cB5677a7C971689228EC835800432B339bA2B](#) and **DNSSECImpl** at [0x0fc3152971714E5ed7723FAFa650F86A4BaF30C5](#) — for any future BANKON DNS-import work (e.g. claiming [bankon.io](#) or [bankon.app](#) on-chain). [GitHub](#) [GitHub](#)

.eth pricing rules locked in by the StablePriceOracle (May 2026)

ENS pricing has been stable across the last several controller upgrades and continues to use a Chainlink ETH/USD oracle to convert USD-denominated fees to ETH at registration time. The

current schedule documented in the [ENS .eth Registrar docs](#) and the [support article on fees](#) is: **5+ character names cost \$5.00/year, 4-character names cost \$160/year, and 3-character names cost \$640/year**, with 1- and 2-character names disabled at the protocol level. Because `bankon` is six characters, the rent will be exactly \$5/year denominated in USD, paid in ETH; an integration is expected to send 3-5% over `rentPrice()` to absorb mid-transaction ETH/USD volatility, and any overpayment is refunded automatically. The grace period is `90 days`, after which the name enters the 21-day exponential premium auction described above, and only after that does `bankon.eth` become available to the public again — meaning a single registration locks in the namespace for $365 + 90 + 21 = 476$ days minimum before any attacker could obtain it. [GitHub +7](#)

The commit/reveal flow has timing parameters `minCommitmentAge = 60 seconds` and `maxCommitmentAge = 24 hours`. A typical registration costs around 250k-350k gas if it also wraps and sets initial records; at 1-2 gwei post-Fusaka, that's around \$0.03-\$0.10 in gas — confirming that the BANKON markup, not gas, will be the dominant cost. [Ens](#) [GitHub](#)

Fuses model for "unruggable" subnames

The full enumeration of fuses from [ens-contracts/contracts/wrapper/README.md](#) and the live contract is: `CANNOT_UNWRAP = 1`, `CANNOT_BURN_FUSES = 2`, `CANNOT_TRANSFER = 4`, `CANNOT_SET_RESOLVER = 8`, `CANNOT_SET_TTL = 16`, `CANNOT_CREATE_SUBDOMAIN = 32`, `CANNOT_APPROVE = 64`, `PARENT_CANNOT_CONTROL = 1 << 16 = 65536`, `IS_DOT_ETH = 1 << 17 = 131072`, `CAN_EXTEND_EXPIRY = 1 << 18 = 262144`. The constants `PARENT_CONTROLLED_FUSES = 0xFFFF0000` and `USER_SETTABLE_FUSES = 0xFFFDFFFF` define the bit ranges that the parent vs. owner can set. The critical rule for BANKON is "to burn fuses on a name, the parent name must be Locked" — meaning `bankon.eth` itself must first have `CANNOT_UNWRAP` burned, after which the BankonSubnameRegistrar can burn `PARENT_CANNOT_CONTROL | CANNOT_UNWRAP` on `alice.bankon.eth` at mint time and produce a truly Emancipated subname that BANKON itself cannot revoke. This is the security primitive that justifies charging more than zero per subname. [Mirror](#)

The three subname-issuance models, scored

Model A — L1 NameWrapper subdomains is the recommended path. The mint cost at current post-Fusaka conditions is well under \$0.20 per subname, the subname is a real ERC-1155 NFT that appears in wallets and marketplaces, fuses provide cryptographic unruggability, and there is no off-chain trust assumption. The downside is per-mint gas; at moderate scale (under ~50k subnames/year), L1 economics are clearly best.

Model B — L2 / off-chain via CCIP-Read has converged around two production stacks.

NameStone's [Durin framework](#) is the open-source path: users point their `.eth` resolver at the **Durin L1 Resolver** at `@0x8A968aB9eb8C084FBC44c531058Fc9ef945c3D61` and call `setL2Registry(registryAddress, chainId)`. The L2 Registry is deployed via the `L2RegistryFactory` at the CREATE2-vanity address

`0xDdddddDdDDD8Aa1f237b4fa0669cb46892346d22d` — the same address on Base, Optimism, Arbitrum, Linea, Polygon, Celo, Scroll, and all their Sepolia/Amoy testnets, supported by a `UniversalSigValidator` at `0x164af34fAF9879394370C7f09064127C043A35E9`. NameStone also publishes a separate hosted L1 resolver at `0xA87361C4E58B619c390f469B9E6F27d759715125` for their fully off-chain API product. `Namespace` (now also at `namespace.tech`) is the API/dashboard alternative, charging a fixed **5% fee on subname minting revenue** only when the parent sets a non-zero price, and as of 2026 manages over 800,000 subnames across 30+ partners "including Celo, Filecoin, POAP, and PinMe" per its own website. [GitHub + 4](#)

Model C — Hybrid L1 parent / L2 issuer uses ENSIP-10 wildcard resolution and EIP-3668 CCIP-Read. The L1 resolver attached to `bankon.eth` reverts with `OffchainLookup(address sender, string[] urls, bytes callData, bytes4 callbackFunction, bytes extraData)`; the client (viem/wagmi/ethers) fetches from BANKON's gateway, returns the result with a Merkle or EIP-712 proof, and the L1 resolver verifies before returning. Coinbase's `.base.eth` is the largest live example — Basenames surpassed 400,000 registrations within approximately two days of its August 2024 launch, with Base lead Jesse Pollak announcing 200,000 registrations in the first week alone. The reference implementation Gregory should clone from is `ensdomains/offchain-resolver` and the offchain ENS gateway example at `gskril/ens-offchain-registrar` cited in the ENS subdomains docs. [Ens + 3](#)

Basenames as a comparative datapoint (and a parallel-deployment option)

`Basenames` is the most successful ENS-on-L2 deployment to date and ships a Coinbase-built fork of the ENS contracts on Base (chain `eip155:8453`). The verified Basescan addresses are: Registry at `0xb94704422c2a1e396835a571837aa5ae53285a95`, BaseRegistrar at `0x03c4738Ee98aE44591e1A4A4F3CaB6641d95DD9a`, original RegistrarController at `0x4cCb0BB02FCABA27e82a56646E81d8c5bC4119a5` (now superseded for active use by an `UpgradeableRegistrarController` proxy at `0xa7d2607c6BD39Ae9521e514026CBB078405Ab322` with implementation `0x9ad14968093C5E8C2A8CC86f6868CFEE8C659717`), L2Resolver at `0xC6d566A56A1aFf6508b41f6c90ff131615583BCD` (upgradeable proxy at `0x426fA03fB86E510d0Dd9F70335Cf102a98b10875` impl `0xf74b949f2105178eEEd4Ef35a131715E967337ab`), ReverseRegistrar at `0x79ea96012eea67a83431f1701b3dff7e37f9e282`, and the L1 resolver for `base.eth` on Ethereum mainnet at `0xde9049636F4a1dfE0a64d1bFe3155C0A14C54F31`. Basename pricing is ETH-denominated rather than USD-pegged: **0.1 ETH/yr for 3-char, 0.01 ETH/yr for 4-char, 0.001 ETH/yr for 5-9 char, and 0.0001 ETH/yr for 10+ char**. For BANKON, the practical implication is that if you want a free `.base.eth` identity for marketing alongside the canonical `bankon.eth`, you can register `bankon.base.eth` at 0.001 ETH/year on Base via the Coinbase RegistrarController. [github + 5](#)

Linea, Polygon, Optimism, Arbitrum and the rest

`Linea ENS` is a forked-from-`ensdomains/ens-contracts` deployment by Consensus, gated by

Proof-of-Humanity via Verax with `FixedPriceOracle` set extremely high to force the free-with-PoH path; the namespace is `name.linea.eth` and the resolution path is L1-anchored CCIP-Read. Optimism, Arbitrum One, Scroll and Linea now host an `L2ReverseRegistrar` deployed at a unified address via the ENS Labs CREATE3 process (Safe `0x353530FE74098903728Ddb66Ecdb70f52e568eC1` mainnet, `0x343431e9CEb7C19cC8d3eA0EE231bFF82B584910` testnet) — these are the targets for ENSIP-19 L2 primary names. Polygon's competitor is **Unstoppable Domains UNSRegistry** at `0xa9a6a3626993d487d2dbda3173cf58ca1a9d9e9f` (implementation `0x11227e54f19934164a81d5add1ce5825d46b2271`), MintingManager at `0x7be83293BeeDc9Eba1bd76c66A65F10F3eFaeC26`, and ProxyReader at `0x423F2531bd5d3C3D4EF7C318c2D1d9BEDE67c680`, with source in `unstoppabledomains/uns`; Unstoppable's lifetime-ownership model and proprietary TLDs (`.crypto`, `.nft`, `.x`, `.wallet`, `.bitcoin`, `.dao`, `.888`, `.blockchain`) are incompatible with ENS resolution and should be treated as a separate identity track BANKON does not need to integrate at v1. [GitHub + 6](#)

Algorand NFDomains integration (NFD V3) for BANKON's Algorand rail

NFDomains V3 introduces direct, permissionless minting from the registry contract, expiration of registrations (max 20 years), and box storage for per-address NFD lookups. The verified Algorand mainnet **Registry Application ID** is `760937186`, testnet is `84366825`, and betanet is `842656530`, per [docs.nf.domains/smart-contracts/resolving-nfds/nfd-registry](#). The current deployed contract version is 3.9 per the [Contract Changelog](#). The TypeScript SDK is at `TxnLab/nfd-sdk` with `NfdClient.mainNet()` and `NfdRegistryId.MAINNET`. For BANKON, the meaningful integration point is that each `bankon.eth` subname's resolver should optionally write the holder's Algorand NFD into a TEXT record (e.g., `text(node, "algorand.nfd") = "alice.algo"`) so that cross-chain identity is mutually discoverable from either direction.

[Medium + 4](#)

x402-AVM settlement plumbing (the BANKON-distinctive rail)

The `GoPlausible x402-avm` packages implement the x402 HTTP 402 Payment Required protocol over the Algorand Virtual Machine. The critical constants the BANKON registrar must hard-code or load are: `ALGORAND_MAINNET_CAIP2 = "algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8="`, `ALGORAND_TESTNET_CAIP2 = "algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI="`, `USDC_MAINNET_ASA_ID = 31566704`, `USDC_TESTNET_ASA_ID = 10458941`, with USDC decimals fixed at 6. The reference facilitator URL is <https://x402.org/facilitator>. The Python package is installed as `pip install "x402-avm[fastapi,avm]"` and imports from the `x402` namespace; TypeScript packages live under `@x402-avm/avm`, `@x402-avm/core`, `@x402-avm/express`, `@x402-avm/next`, `@x402-avm/hono`, and `@x402-avm/paywall`. As of v2.6+ the package no longer depends on `algosdk`; it uses `@algorandfoundation/algokit-utils@10.0.0-alpha.39` internally. The settlement flow for BANKON is: the user signs a USDC `AssetTransferTxn` on Algorand → the

facilitator verifies and settles, returning an x402 receipt → an L1 attestation contract on Ethereum (BANKON-deployed, signed by the facilitator's `AVM_PRIVATE_KEY`) accepts the receipt as proof-of-payment → `BankonSubnameRegistrar.registerWithX402(label, owner, x402Receipt, signature)` validates the EIP-712 signature against the registered facilitator key, marks the receipt nullifier as spent to prevent replay, and mints the subname through `NameWrapper.setSubnodeRecord`. Refund-on-failure must be handled on Algorand because the L1 contract has no power to claw back ASA payments; the cleanest pattern is to delay the Algorand `clawback` only after L1 mint success, using the facilitator as the timed-release escrow. [GitHub + 8](#)

ERC-7857 iNFT layer for AI-agent subnames

ERC-7857 is the OG Labs "Intelligent NFT" extension to ERC-721, designed for AI agents that need encrypted metadata, oracle-verified secure re-encryption on transfer, and dynamic update permissions. The reference implementation is documented at docs.0g.ai/developer-hub/building-on-0g/inft/erc7857; the contract signature is `contract ERC7857 is ERC721, Ownable, ReentrancyGuard` with `mapping(uint256 => bytes32) _metadataHashes`, `mapping(uint256 => string) _encryptedURIs`, and a `validProof` modifier gated by `IOracle(oracle).verifyProof(proof)` with `PROOF_VALIDITY_PERIOD = 1 hours`. The transfer function takes an extra `bytes calldata sealedKey` argument so the oracle can re-encrypt metadata for the new owner inside a TEE. For BANKON, the integration is straightforward: when a user mints `agent.bankon.eth` with the iNFT toggle on, the BankonSubnameRegistrar deploys (or references) a per-agent ERC-7857 instance on the OG chain, stores the metadata hash and encrypted URI pointer in a TEXT record on the BankonSubnameResolver, and uses the agent's ERC-6551 token-bound account address as the canonical `addr(node)` resolution. [Mitosis University + 4](#)

ERC-6551 token-bound accounts for per-subname agent wallets

ERC-6551 went live on Ethereum mainnet on May 7, 2023; per [EIP-6551 on eips.ethereum.org](https://eips.ethereum.org/EIP-6551), the standard was authored by Jayden Windle, Benny Giang (co-author of ERC-721 and co-creator of CryptoKitties), Steve Jang, and colleagues at Future Primitive, with the initial proposal posted February 23, 2023. The singleton registry is deployed at the same address on every EVM chain: `0x000000006551c19487814612e58FE06813775758`, with source at [erc6551/reference](#). The two functions BANKON's registrar must call are `createAccount(address implementation, bytes32 salt, uint256 chainId, address tokenContract, uint256 tokenId)` and the deterministic `account(...)` view that returns the CREATE2 address without deployment. Because NameWrapper is ERC-1155 rather than ERC-721, BANKON should ship a thin ERC-721 "AgentNameWrapper" adapter (one NFT per subname) so the canonical ERC-6551 registry can bind the agent wallet — or use the recently audited ERC-6551 implementations that have been extended to accept ERC-1155 token IDs. Either way, the resulting `agent.bankon.eth` resolves to a deterministic smart-contract wallet that can hold assets, sign transactions, accumulate

reputation, and participate in BONAFIDE attestation without leaking the human owner's EOA.

Thirdweb + 6

Details

Building `BankonSubnameRegistrar.sol`

The core contract is a NameWrapper-aware registrar that holds the wrapped `bankon.eth` ERC-1155 token and exposes a `register(string label, address owner, uint32 fuses, uint64 expiry, bytes32 referrer)` function. Deployment sequence: deploy the contract; transfer the wrapped `bankon.eth` to it via `NameWrapper.safeTransferFrom`; or alternatively call `NameWrapper.setApprovalForAll(registrarAddress, true)` so the registrar can mint on behalf of a Safe-held parent. The first approach is more secure (the registrar literally owns the parent and cannot be revoked except via a fuse-protected admin path), but it means rotating the registrar requires unwrapping which destroys all fuse guarantees on children; the approval-only approach lets a Safe rotate the registrar without affecting children, at the cost of trusting the Safe.

Functionally the register flow inside the contract does the following in order. First it validates the label using the same ASCII/UTS-46 normalization rules ENS does — the simplest path is to require the caller to pass a pre-normalized string and reject anything that hashes differently from the canonical form. Second it computes the price by combining a base-fee (configurable per-character-length, allowing BANKON to mimic the ENS 5/160/640 tiers or to offer flat \$1/year subnames), the BANKON markup, the AgenticPlace listing add-on if requested, and the parent-domain-owner share if this is a BankonDomainHosting flow. Third it accepts payment in one of three rails: `msg.value` for native ETH (converted to the registrar's USD-denominated price via a Chainlink ETH/USD feed at `0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419`), `IERC20(USDC).permit(...)` followed by `transferFrom` for USDC on L1 (using Circle's USDC at `0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48`), or `_validateX402Receipt(bytes32 receiptHash, bytes signature)` which checks that the BANKON-trusted x402 facilitator key signed the receipt and that the receipt has not been replayed. Fourth it calls `NameWrapper.setSubnodeRecord(parentNode, label, owner, resolver, ttl, fuses, expiry)` with `fuses = PARENT_CANNOT_CONTROL | CANNOT_UNWRAP | (optionalUserFuses & USER_SETTABLE_FUSES)` and an expiry not exceeding `bankon.eth`'s own expiry. Fifth it emits a structured event `SubnameMinted(bytes32 indexed parentNode, bytes32 indexed labelhash, address indexed owner, uint256 ethFee, uint256 usdcFee, bytes32 x402Receipt, address tba, address inft)` so AgenticPlace's indexer can pick it up without polling. The renewal flow is symmetric, calling `NameWrapper.setChildFuses(parentNode, labelhash, fuses, newExpiry)` to extend.

The fee-split implementation should use a single `_distributeFees(uint256 ethAmount, uint256 usdcAmount, bytes32 listingFlags)` internal function that reads `treasury`, `agenticPlacePool`, `parentOwnerPayout` (only set in the BankonDomainHosting path) from immutable or upgradeable storage and pushes funds in one transaction; for production, settlement should use pull-payments via OpenZeppelin's `PaymentSplitter` pattern so a single recipient bricking transfers cannot block a mint.

Building `BankonDomainHosting.sol`

This meta-contract lets external `.eth` owners — say, `vitalik.eth` — list their name with BANKON to issue and sell subdomains through the same rails. The flow is: domain owner calls `enrollDomain(bytes32 node, uint256 listingFee, RevSplit memory split, bool agenticPlaceListed)` paying a one-time `LISTING_SETUP_FEE` (configurable, perhaps 0.05 ETH); the contract validates ownership via `NameWrapper.ownerOf(uint256(node)) == msg.sender` for wrapped names or `ENSRegistry.owner(node) == msg.sender` for unwrapped; the owner must then call `NameWrapper.setApprovalForAll(BANKON_HOSTING_ADDRESS, true)` (or send the wrapped NFT to the hosting contract for full custody); the contract stores the per-domain configuration including the owner's chosen subname pricing curve, the BANKON markup percentage (default 10%), the AgenticPlace listing toggle, and an optional iNFT-by-default flag. Subsequent calls to `registerSub(bytes32 parentNode, string label, ...)` execute exactly like the BankonSubnameRegistrar primary flow but route the domain-owner share to the enrollment record. Critically, the hosting contract must enforce that the parent name remains Locked (CANNOT_UNWRAP burned) before it allows any fuse-burning subname mints, otherwise the domain owner could rug all subname holders by unwrapping — this is implemented by `require(NameWrapper.allFusesBurned(parentNode, CANNOT_UNWRAP), "ParentNotLocked")`.

Building `BankonSubnameResolver.sol`

The resolver inherits the `PublicResolver` base and adds: a `MINDX_ENDPOINT_KEY = "mindx.endpoint"` text-record convention pointing at `https://mindx.pythai.net/api/agent/<subname>`, a `BONAFIDE_ATTESTATION_KEY = "bonafide.attestation"` storing the keccak256 of the latest BONAFIDE attestation contract address, a `CAPABILITY_KEY = "agent.capabilities"` for free-form JSON capability descriptors, a `INFT_POINTER_KEY = "inft.uri"` for the encrypted metadata URI, and an automatic `addr(node) = tokenBoundAccount(node)` override when the iNFT toggle is on. For CCIP-Read (only if BANKON later adopts an L2 mirror), the resolver implements `IExtendedResolver.resolve(bytes calldata name, bytes calldata data)` and reverts with `OffchainLookup(address(this), [BANKON_GATEWAY_URL], data, this.resolveCallback.selector, abi.encode(name, data))`.

The recommended layout under the flat snake_case standard is:

```
foundry.toml
src/
  bankon_subname_registrar.sol
  bankon_domain_hosting.sol
  bankon_subname_resolver.sol
  bankon_x402_attestor.sol
  bankon_inft_adapter.sol
test/
  bankon_subname_registrar_test.sol
  bankon_domain_hosting_test.sol
  bankon_subname_resolver_test.sol
  bankon_x402_settlement_test.sol
  bankon_inft_test.sol
fork/
  mainnet_registration_fork_test.sol
script/
  deploy.s.sol
  configure_fuses.s.sol
```

`foundry.toml` should set `[profile.default] solc = "0.8.26"`, `optimizer = true`, `runs = 200`, `fuzz.runs = 5000`, `invariant.runs = 256`, `invariant.depth = 32`; `[rpc_endpoints]` `mainnet = "${MAINNET_RPC_URL}"`, `base = "${BASE_RPC_URL}"`; and `[etherscan] mainnet = { key = "${ETHERSCAN_API_KEY}" }`. Fork tests should call

`vm.createSelectFork(vm.envString("MAINNET_RPC_URL"), 22_500_000)` (pick a block known to have all current ENS deployments). The unit tests should mock the ENS contracts via `MockENS`, `MockNameWrapper`, `MockPriceOracle` stubs in `test/mocks/`, while fork tests exercise the real `ETHRegistrarController` at `0xf3cE5D01e0f33f41DbB39035dB9745962F1f968` and real `NameWrapper` at `0x0635513f179D50A207757E05759CbD106d7dFcE8` against a forked mainnet, registering a throwaway test name to verify the full flow. Differential tests compare BANKON's price-quote logic against the live `StablePriceOracle.price(name, expires, duration)`. Coverage target: 95% line, 100% branch on payment splits and fuse-validation paths.

Mainnet deployment plan

The sequence is: (1) deploy `BankonSubnameResolver` first (no dependencies); (2) deploy `BankonX402Attestor` with the facilitator's public key; (3) deploy `BankonSubnameRegistrar` taking the resolver, attestor, treasury Safe, and NameWrapper addresses as constructor args (use CREATE2 via `0x4e59b44847b379578588920cA78FbF26c0B4956C` for a vanity `0xBANK...` prefix if desired); (4) deploy `BankonDomainHosting` referencing the registrar; (5) execute the registration

of `bankon.eth` through the Safe by calling `ETHRegistrarController.commit(commitment)`, waiting at least 60 seconds, then `ETHRegistrarController.register(name, owner, duration, secret, resolver, data, reverseRecord, referrer)`; (6) call `NameWrapper.wrapETH2LD("bankon", safeAddress, 0, resolverAddress)` if not already wrapped during register; (7) call `NameWrapper.setFuses(node, CANNOT_UNWRAP)` to Lock the parent; (8) `NameWrapper.safeTransferFrom(safe, registrarAddress, uint256(node), 1, "")` to give the registrar full custody. Each step should be a separate Safe transaction (Gregory's BONAFIDE Safe pattern), with multisig review between steps. Estimated total gas: roughly 1.2M for contract deploys plus 800k for the registration/wrap/lock sequence — at 2 gwei and roughly 2M total gas, that's ~0.004 ETH ≈ \$8.43 at the ETH price of \$2,107.05 quoted on Bybit for May 19, 2026. Verify on Etherscan with `forge verify-contract --etherscan-api-key $ETHERSCAN_API_KEY --chain mainnet --watch` immediately after each deploy, and additionally upload to Sourcify with `forge verify-contract --verifier sourcify` for redundancy. RareSkills Bybit

Chain-ID mapping for `agenticplace.pythai.net/allchain.html`

The mapping the AgenticPlace loader needs to register is: Ethereum mainnet `eip155:1`, Base `eip155:8453`, Optimism `eip155:10`, Arbitrum One `eip155:42161`, Linea `eip155:59144`, Polygon `eip155:137`, Scroll `eip155:534352`, zkSync Era `eip155:324`, Mantle `eip155:5000`, Blast `eip155:81457`, Algorand mainnet via CAIP-2 `algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkIt8=`, Algorand testnet `algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI=`, and Gregory's Arc Testnet `eip155:5042002`. The OG chain identifiers should be added once OG Labs publishes a finalized CAIP-2 string; until then the loader should fall back to whatever the OG docs list at docs.og.ai.

GitHub + 3

Documentation URLs Gregory should bookmark

The authoritative ENS documentation is at docs.ens.domains with critical sections at docs.ens.domains/registry/eth for the .eth registrar, docs.ens.domains/wrapper/contracts for NameWrapper, docs.ens.domains/wrapper/fuses for fuse semantics, docs.ens.domains/web/subdomains for the subname-registrar guides, docs.ens.domains/resolvers/ccip-read for CCIP-Read, docs.ens.domains/ensip/23 for the Universal Resolver, docs.ens.domains/learn/deployments for the current address table, and the canonical [ENS Contract Deployments wiki](#). EIP specifications: [EIP-3668 CCIP-Read](#), [EIP-6551 Token Bound Accounts](#), the in-progress [ERC-7857](#) on OG docs. Adjacent stacks: [Durin docs](#), [Namespace docs](#), [Basenames repo](#), [Basenames FAQ](#), [NFD on-chain reference](#), [GoPlausible x402-avm](#), and the [x402 specification](#) at x402.org.

Recommendations

Stage 1 (week 1) — secure `bankon.eth` on L1. Register `bankon.eth` immediately through the Safe via the current ETHRegistrarController at `0xfb3cE5D01e0f33f41DbB39035dB9745962F1f968`, for at least 5 years prepaid (\$25 USD-equivalent in ETH, locking in the \$5/yr price). Wrap into the NameWrapper at `0x0635513f179D50A207757E05759CbD106d7dFcE8`, but **do not yet burn CANNOT_UNWRAP** — keep recovery flexibility until the registrar is audited. Set the resolver to the canonical PublicResolver at `0x231b0Ee14048e9dCcD1d247744d114a4EB5E8E63` until the BANKON resolver is live. Estimated effort: 2-4 hours of Safe operations plus ~\$10 in gas + \$25 in name rent.

Stage 2 (weeks 2-4) — build and audit the core contracts. Implement `BankonSubnameRegistrar`, `BankonSubnameResolver`, and `BankonX402Attestor` against the Foundry layout above. The decision threshold for whether to also build `BankonDomainHosting` in v1 is whether you can name two external `.eth` owners committed to listing in the first 30 days; if not, defer hosting to v2 to reduce audit surface. Engage one auditor (Spearbit, Code4rena spot audit, or OpenZeppelin) — budget ~\$30k-\$60k. Deploy to Sepolia in parallel using ENS Sepolia deployments listed in [ens-contracts wiki](#) for end-to-end testnet validation.

Stage 3 (week 5) — mainnet deploy and lock. Run the eight-step deployment sequence above. Only after one full audit cycle and a successful Sepolia trial should you call `NameWrapper.setFuses(node, CANNOT_UNWRAP)` to lock `bankon.eth` — this is irreversible and unlocks the ability to issue truly-unruggable child subnames. Transfer the wrapped parent to the registrar contract.

Stage 4 (weeks 6-8) — minimal reactive UI. Build `BankonRenewalUI` and `BankonDomainOwnerUI` as Next.js single-page apps with wagmi v2 + viem v2 + RainbowKit, using the UniversalResolver at `0xEeEeEeE14D718C2B47D9923Deab1335E144EeEe` for all read paths. Implement the three payment rails: native ETH (direct contract call), USDC via `permit` (Circle USDC `0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48`), and the x402 Algorand flow via `@x402-avm/next` and the BANKON facilitator. The UI must include a single-click "iNFT mint" toggle that, when enabled, deploys the ERC-7857 metadata pointer and derives the ERC-6551 token-bound account address before final mint.

Stage 5 (week 9+) — AgenticPlace, mindX, BONAFIDE wiring. Wire the `SubnameMinted` event into the AgenticPlace indexer so listing cards are auto-created. Register each subname's `mindx.endpoint` text record pointing at `https://mindx.pythai.net/api/agent/<labelhash>`. Add BONAFIDE attestation gating to the registrar's `register()` function as an optional modifier (initially off; enable once nine-contract Latin-named protocol is in production).

Threshold to revisit L2 mirror (Stage 6, conditional). If L1 gas costs rise above 30 gwei sustained for two weeks AND BANKON mint volume exceeds 5,000 subnames per month, deploy a Durin L2Registry via `L2RegistryFactory` at

`0xDdddDdDDD8Aa1f237b4fa0669cb46892346d22d` on Base and Optimism, swap the BankonSubnameResolver to the Durin L1Resolver pattern via `setL2Registry()` on the Durin L1 resolver at `0x8A968aB9eb8C084FBC44c531058Fc9ef945c3D61`, and gate new subnames behind a feature flag. Below those thresholds, L2 mirror adds operational complexity without meaningful cost reduction.

Caveats

The ENSv2 architectural roadmap is still in flight as of May 2026; while ENS Labs has committed to L1-only deployment, the planned hierarchical-registries upgrade and new "personalized registries" feature will affect how BANKON should issue subnames once it ships. Re-read ens.domains/blog/post/ens-staying-on-ethereum and the [ENSv2 project plan](#) at deploy time. The NameWrapper address `0x0635513f179D50A207757E05759CbD106d7dFcE8` is what the official ens-contracts wiki publishes as of March 2026; earlier ENS proposal documents (notably EP3.5) referenced `0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401` — verify on Etherscan at deploy time that the wiki address resolves to the live wrapper before locking BANKON's registrar to it. The current `ETHRegistrarController` at `0xfb3cE5D01e0f33f41DbB39035dB9745962F1f968` includes a `referrer` parameter in its `NameRegistered` event but the actual referrer-program economics (revenue share to BANKON if you put BANKON's bytes32 in the referrer slot of registrations you facilitate) are governed off-chain through the [namehash/ens-referrals](#) program — confirm enrollment status separately. ERC-7857 is still moving toward final EIP status; the OG reference implementation is production-grade but the standard itself may evolve, so the BankonInftAdapter should be written as a thin pluggable layer that can swap implementations without touching the registrar core. NFD V3 contract version 3.9 is the current deployed major release per the changelog, but Algorand contracts update more aggressively than EVM contracts — pin the version your SDK targets and watch the [NFD Contract Changelog](#). The x402-AVM stack underwent a breaking v2→v2.6 change removing `algosdk` as a direct dependency in favor of `@algorandfoundation/algokit-utils@10.0.0-alpha.39`; ensure all BANKON Python services pin `x402-avm[avm]>=2.6` to avoid the older API surface. The Basenames RegistrarController has two live versions (the original `0x4cCb...9a5` and the new `UpgradeableRegistrarController` proxy `0xa7d2...322`) — for any parallel deployment of `bankon.base.eth`, use the proxy, not the original. [CoinMarketCap](#) [GitHub](#)